Guía de estudio: Tokenización, Semántica vectorial y Embeddings

Preparación para práctica y quiz — PLN con Deep Learning

Raúl Andrés Alzate Gómez

2026

El continuum del PLN

Todo el curso recorre una misma idea: **convertir lenguaje en vectores donde la geometría codifica el significado**. El texto pasa por 5 etapas; cada una resuelve una pregunta y alimenta a la siguiente.

1

Tokenización

¿Cómo partir el texto en unidades?

char · palabra · subpalabra · byte

Ej. `"gatos"` → `gato` + `##s`

2

Representación dispersa

¿Cómo comparar documentos?

TF · TF-IDF · coseno

Ej. un documento → vector de 10 000 números, casi todos en cero

3

Embeddings densos

¿Cómo capturar significado en pocos números?

Word2Vec · GloVe · FastText

Ej. `rey` → vector denso de 300 números

4

Embeddings contextuales

¿Cómo distinguir significados según el contexto?

ELMo BERT/BETO GPT

Ej. `banco` (dinero) ≠ `banco` (asiento)

5

Embeddings de oración

¿Cómo comparar frases completas?

SBERT MiniLM bge-m3

Ej. `“el gato duerme”` ≈ `“el felino descansa”`

Por qué cada salto: cada técnica nace para arreglar una falla de la anterior.

Salto	Falla que arregla
Crudo → Tokenización	El modelo necesita unidades discretas y controlar palabras nuevas (OOV)
TF → TF-IDF	Las palabras muy frecuentes (`el` , `de`) no aportan y distorsionan
TF-IDF → Word2Vec	Vectores de miles de ceros → densos y cortos que capturan semántica
Word2Vec → FastText	Palabras nuevas y morfología del español (`caminar` / `caminé`)
Estático → BERT	Una palabra tiene un solo vector aunque signifique cosas distintas (polisemia)
BERT → SBERT	Comparar oraciones con BERT crudo es lentísimo

Glosario de siglas y términos

Casi todo en PLN se nombra con siglas en inglés. Esta tabla las traduce y explica en una frase. Úsala como referencia mientras lees el resto de la guía.

Generales

Sigla	Significa (inglés)	Qué es (en simple)
PLN / NLP	(Natural Language Processing)	Procesamiento de Lenguaje Natural: que la máquina entienda/genere texto
IA / AI	(Artificial Intelligence)	Inteligencia Artificial
LLM	Large Language Model	Modelo de lenguaje grande (GPT, LLaMA…)
OOV	Out-Of-Vocabulary	Palabra “fuera de vocabulario”: no se vio al entrenar
token	—	Unidad mínima de texto (palabra, subpalabra o carácter)
corpus	—	Colección grande de textos para entrenar
embedding	—	Vector de números que representa texto

Tokenización

Sigla	Significa	Qué es
BPE	Byte-Pair Encoding	Fusiona los pares de caracteres más frecuentes
WordPiece	—	Como BPE, pero fusiona lo que más sube la probabilidad; usa `##`
Unigram	—	Parte de un vocab grande y poda lo inútil
EM	Expectation-Maximization	Algoritmo que ajusta probabilidades iterando
wwm	Whole Word Masking	Enmascara palabras completas al entrenar (BETO usa esto)
cased	—	Distingue mayúsculas/minúsculas (`Casa` ≠ `casa`)

Sigla	Significa	Qué es
SBW	Spanish Billion Words	Corpus estándar en español (~1 800 M de tokens)

Semántica vectorial clásica

Sigla	Significa	Qué es
VSM	Vector Space Model	Representar documentos como vectores
TF	Term Frequency	Cuántas veces aparece una palabra en un documento
IDF	Inverse Document Frequency	Qué tan rara/informativa es una palabra
TF-IDF	TF × IDF	Frecuente aquí **y** rara en el resto
L1 / L2	—	Normas: L1 = suma de absolutos; L2 = raíz de cuadrados (el “largo”)
IR	Information Retrieval	Recuperación de información (buscadores)

Sigla	Significa	Qué es
PCA	Principal Component Analysis	Reduce dimensiones conservando la varianza
t-SNE	t-distributed Stochastic Neighbor Embedding	Reduce dimensiones conservando vecindades
LSA	Latent Semantic Analysis	Método clásico de temas por matriz de conteos
PPMI	Positive Pointwise Mutual Information	Peso de co-ocurrencia entre palabras

Embeddings y modelos

Sigla	Significa	Qué es
CBOW	Continuous Bag-Of-Words	Word2Vec que predice la palabra desde el contexto
Skip-gram	—	Word2Vec que predice el contexto desde la palabra
GloVe	Global Vectors	Embeddings desde co-ocurrencia global
FastText	—	Word2Vec + n-gramas de caracteres (maneja OOV)
BERT	Bidirectional Encoder Representations from Transformers	Modelo contextual bidireccional
BETO	—	El BERT entrenado en español
MLM	Masked Language Modeling	Tarea: adivinar palabras tapadas

Sigla	Significa	Qué es
NSP	Next Sentence Prediction	Tarea: ¿esta frase sigue a la otra?
ELMo	Embeddings from Language Models	Primer embedding contextual (con LSTM)
GPT	Generative Pre-trained Transformer	LLM generativo (decoder)
SBERT	Sentence-BERT	BERT adaptado para comparar oraciones por coseno
STS	Semantic Textual Similarity	Tarea de medir parecido entre frases
MPNet / MiniLM	—	Arquitecturas eficientes para embeddings de oración
bge-m3	BAAI General Embedding (m3)	Modelo multilingüe top para recuperación

Aplicaciones

Sigla	Significa	Qué es
NER	Named Entity Recognition	Detectar entidades (personas, fechas, enfermedades …)
IOB	Inside-Outside-Begin	Esquema de etiquetas: `B_` inicio, `I_` dentro, `O` fuera
RAG	Retrieval-Augmented Generation	Buscar contexto y dárselo a un LLM para responder
FAISS	Facebook AI Similarity Search	Librería para buscar vectores similares a gran escala

PARTE I — Tokenización

1.1 Qué es la tokenización y por qué importa

La **tokenización** es el primer eslabón de cualquier pipeline de PLN: convierte una secuencia continua de caracteres en una lista discreta de **tokens** que el modelo manipula como vectores.

La elección del esquema condiciona cuatro cosas:

- **Tamaño de la matriz de embeddings** (vocabulario × dimensión).
- **Longitud media de la secuencia** (la atención del Transformer es cuadrática en la longitud).
- **Capacidad de manejar palabras desconocidas** (OOV, *out-of-vocabulary*) y morfología.
- **Transferencia entre idiomas** (cobertura Unicode, scripts no latinos).

1.2 Familias de algoritmos

Estrategia	Unidad	Ventaja	Desventaja
Carácter	1 char	Sin OOV, vocab mínimo (~100)	Secuencias muy largas, pierde semántica de palabra
Palabra	palabra completa	Semántica directa	Vocab enorme, OOV severo, mal con flexión
Subpalabra	morfemas / fragmentos	Balance: poco OOV + secuencias razonables	Requiere entrenamiento
Byte	1 byte UTF-8	Universal multi-idioma	Secuencias aún más largas que char

Los modelos *state-of-the-art* (BERT, GPT, T5, LLaMA) usan **subpalabra**. Ejemplo: la palabra `tokenización` segmentada por cada estrategia:

Estrategia	Resultado	Nº tokens	Comentario
Carácter	`t o k e n i z a c i ó n`	12	Secuencia larguísima
Palabra	`tokenización`	1	`<unk>` si no estaba en el vocab
BPE	`token` + `ización`	2	Fusiones por frecuencia
WordPiece	`token` + `##ización`	2	Fusiones por verosimilitud
Unigram	`token` + `ización` *(o)* `tokeni` + `zación`	2	Varias segmentaciones posibles

1.3 Byte-Pair Encoding (BPE)

Algoritmo de compresión adaptado a PLN por **Sennrich, Haddow & Birch (2016)**.

Algoritmo (4 pasos):

1. Inicializa el vocabulario con **todos los caracteres** del corpus.

2. Cuenta la frecuencia de **pares adyacentes** en el texto tokenizado por carácter.
3. Fusiona el par **más frecuente** en un símbolo nuevo.
4. Repite 2–3 hasta alcanzar el tamaño de vocabulario objetivo.

Propiedades: determinista (mismo corpus + tamaño → mismas fusiones), *greedy* (nunca revisa decisiones previas), vocabulario abierto (cualquier palabra es representable, en el peor caso como secuencia de caracteres).

Ejemplo. Corpus `low low lowest newer wider`. Tras varias fusiones BPE produce piezas como `low`, `er`, `est`, `new`, `wid`. La palabra OOV `lowering` se segmenta como `low + er + ing`.

Modelos que lo usan: **GPT-2/3/4, RoBERTa, LLaMA**.

1.4 WordPiece — el de BERT y BETO

Variante de BPE (**Schuster & Nakajima, 2012**; popularizada por **BERT, 2018**).

Diferencia clave con BPE: en vez de fusionar el par *más frecuente*, fusiona el par que **maximiza**

la verosimilitud del corpus bajo un modelo unigrama:

$$\text{score}(A, B) = \frac{P(AB)}{P(A) \cdot P(B)}$$

🟦 **EN PALABRAS**

"¿Vale la pena pegar la pieza A con la pieza B?". Compara qué tan seguido aparecen **juntas** ($P(AB)$) contra lo que se esperaría si fueran **independientes** ($P(A) \cdot P(B)$).

- $P(AB)$ = probabilidad de ver `AB` pegadas.
- $P(A)$, $P(B)$ = probabilidad de cada pieza por separado.

✏️ **EJEMPLO**

Si `ción` casi siempre aparece tras `iza` (`organiza+ción` , `realiza+ción`), entonces $P(\text{ización})$ es alto comparado con $P(\text{iza}) \cdot P(\text{ción})$ → score alto → conviene fusionarlas. Si dos piezas solo coinciden por azar, el score ≈ 1 y no se fusionan. BPE solo miraría la frecuencia bruta; WordPiece mira si la unión **es informativa**.

Notación distintiva: marca las subpalabras *internas* con `##`:

- `tokenization` → `token` , `##ization`

- `playing` → `play`, `##ing`

Tokens especiales de BERT:

Token	Función
`[CLS]`	Marca el inicio; su embedding agregado se usa para clasificación de secuencia
`[SEP]`	Separador de segmentos (par de oraciones, pregunta/contexto)
`[PAD]`	Relleno hasta una longitud fija
`[MASK]`	Posición enmascarada para el preentrenamiento MLM
`[UNK]`	Carácter o palabra fuera del vocabulario

1.5 Modelo Unigram

Propuesto por **Kudo (2018)**; base por defecto de SentencePiece.

Idea: asume un modelo de lenguaje de unigramas sobre subpalabras y entrena las probabilidades con **EM (Expectation-Maximization)**: parte de un vocabulario grande, ajusta las probabilidades $p(x_i)$, y **poda**

iterativamente las subpalabras de menor utilidad hasta llegar al `vocab_size`.

Ventaja: a diferencia de BPE (segmentación única), Unigram puede **muestrear** segmentaciones alternativas → actúa como *data augmentation* (regularización por subpalabra).

Modelos que lo usan: **ALBERT, T5, XLNet, mBART, mT5**.

1.6 SentencePiece

Implementación de Google (**Kudo & Richardson, 2018**) que unifica BPE, Unigram, Word y Char bajo una misma API y resuelve dos problemas prácticos:

1. **Procesa texto en bruto** (sin pretokenizar por espacios). Trata el espacio como un símbolo más (`▁`, U+2581) → funciona en idiomas sin separadores (japonés, chino, tailandés).
2. **Tokenización + destokenización determinista:** la inversa reconstruye el texto exacto.

Operación	Entrada	Salida
`encode`	`"Hola mundo"`	`["_Hola", "_mundo"]`
`decode`	`["_Hola", "_mundo"]`	`"Hola mundo"`

El símbolo `_` (U+2581) marca el espacio que precede a cada palabra; por eso `decode` recupera los espacios exactos.

Parámetros clave: `model_type` (`unigram` / `bpe` / `word` / `char`), `vocab_size` , `character_coverage` (`1.0` para es/en; `0.9995` para zh/ja), `user_defined_symbols` (tokens preservados como `<sep>` , `<cls>`), `input_sentence_size` (subsampling de frases).

1.7 BETO — BERT en español (el del taller)

BETO = `dccuchile/bert-base-spanish-wwm-cased` : un BERT preentrenado en español con tokenizador **WordPiece**. El nombre dice cómo se entrenó: `wwm` = *whole word masking* (tapa palabras completas, no fragmentos) y `cased` = distingue mayúsculas (`Casa` ≠ `casa`). Ejemplo real del taller (frase del dataset biomédico):

```
from transformers import AutoTokenizer
tok =
        AutoTokenizer.from_pretrained("dccuchi
        le/bert-base-spanish-wwm-cased")
tok.tokenize("Abuela materna con cancer de
        mama a los 70 años .")
# →
        ['Abuela','materna','con','cancer','de
        ','mama','a','los','7','##0','años','.
        ']
```

Observa: `70` se parte en `7 ##0` (WordPiece), y al codificar con `tokenizer(...)` se añaden `[CLS]` al inicio y `[SEP]` al final.

1.8 Spanish Billion Words (el corpus)

Corpus de ~**1 800 millones** de tokens en español (Wikipedia + web). ~47 M de oraciones, ~4.7 GB. Es el *estándar* de PLN en español (rol análogo a WikiText-103 en inglés). Por su tamaño, se trocea en archivos y se usa `streaming=True` + `input_sentence_size` al entrenar.

1.9 Tabla comparativa de

algoritmos de tokenización

Criterio	BPE	WordPiece	Unigram	SentencePiece
Criterio de fusión	Par más frecuente	Par que más sube la verosimilitud	Poda por pérdida de likelihood (EM)	Envuelve BPE/Unigram
Dirección	Construye (merge)	Construye (merge)	Reduce (prune)	Según `model_type`
Marcador	(ninguno)	`##` interno	`_` (espacio)	`_` (espacio)
Pretokeniza por espacios	Sí	Sí	Sí	**No** (texto crudo)

Criterio	BPE	WordPiece	Unigram	SentencePiece
Segmentación múltiple	No (única)	No	**Sí** (muestrea)	Si usa Unigram
Maneja idiomas sin espacios	Mal	Mal	Bien	**Excelente**
Modelos	GPT-2/3/4, RoBERTa, LLaMA	BERT, BETO, DistilBERT	ALBERT, T5, XLNet	mT5, NLLB, LLaMA-2

1.10 Ejemplos en Python: tokenizadores

(a) Entrenar y comparar BPE / Unigram con SentencePiece:

```
import sentencepiece as spm

# Entrenar un modelo BPE de 8000 piezas sobre
        un corpus de texto plano
spm.SentencePieceTrainer.train(
    input="corpus_es.txt",
        model_prefix="bpe_es",
    vocab_size=8000, model_type="bpe",
    character_coverage=1.0,                  #
        1.0 para español
    user_defined_symbols=["<cls>", "<sep>"] #
        tokens preservados
)

sp =
        spm.SentencePieceProcessor(model_file=
        "bpe_es.model")
print(sp.encode("tokenización subpalabra",
        out_type=str))
# → ['_token', 'ización', '_sub', 'palabra']
print(sp.encode("tokenización", out_type=int))
        # ids
print(sp.decode(sp.encode("Hola mundo")))
        # reconstruye exacto
print("vocab:", sp.get_piece_size())
```

(b) Comparar nº de tokens entre estrategias:

```
frase = "Las redes neuronales transformer usan
        atención."
for nombre in ["bpe_es.model",
        "unigram_es.model", "word_es.model"]:
    sp =
        spm.SentencePieceProcessor(model_file=
        nombre)
    print(f"{nombre:18s} ->
        {len(sp.encode(frase))} tokens")
# char produce ~7x más tokens que bpe-30k (ver
        Parte VII.B)
```

(c) Tokenizador WordPiece de BETO (HuggingFace):

```
from transformers import AutoTokenizer
tok =
        AutoTokenizer.from_pretrained("dccuchi
        le/bert-base-spanish-wwm-cased")

enc = tok("Abuela con cancer de mama a los 70
        años",
          return_offsets_mapping=True)     #
        offsets para alinear NER
print(tok.convert_ids_to_tokens(enc["input_ids
        "]))
# →
        ['[CLS]','Abuela','con','cancer','de',
        'mama','a','los','7','##0','años','[SE
        P]']
print(enc["input_ids"][:6])
```

IDEA CLAVE

WordPiece parte `70` en `7 ##0` y agrega `[CLS]` / `[SEP]`. En NER esto obliga a **alinear** las etiquetas con los subtokens (ver §4.5).

1.11 Mini-quiz — Tokenización

1. ¿En qué se diferencia WordPiece de BPE en el criterio de fusión?
2. ¿Qué representa el prefijo `##` y qué representa `_` en SentencePiece?
3. ¿Por qué los modelos modernos prefieren subpalabra sobre palabra completa?

4. ¿Para qué sirve `[CLS]` en BERT?
5. ¿Qué ventaja da el modelo Unigram frente a BPE durante el entrenamiento?

(Respuestas en el banco final.)

PARTE II — Semántica vectorial clásica

2.1 Hipótesis distribucional

> ***Harris (1954), Firth (1957):*** *"You shall know a word by the company it keeps." El significado de una unidad lingüística se infiere de los contextos en los que aparece.*

La **semántica vectorial** operacionaliza esa hipótesis representando palabras/documentos como **vectores en** $\mathbb{R}^n$, de modo que la similaridad geométrica refleje similitud semántica.

Generación	Representante	Característica
Clásica (dispersa)	TF, TF-IDF, PPMI, LSA	Vectores largos ($\|V\|$) y mayormente nulos
Escalada (densa)	Word2Vec, GloVe, FastText, BERT	Vectores cortos (50–1024 dim), aprendidos

2.2 Modelo de Espacio Vectorial (VSM)

Propuesto por **Salton, Wong & Yang (1975)**. Tres ingredientes:

1. **Vocabulario** $V = \{t_1, \ldots, t_m\}$.
2. **Matriz término–documento** $X \in \mathbb{R}^{m \times n}$ con $X[i, j]$ = peso del término t_i en el documento d_j.
3. **Función de similaridad** entre columnas (documentos) o filas (términos).

2.3 TF, IDF y TF-IDF

Frecuencia de término (TF): $\mathrm{tf}(t, d)$ = veces que t aparece en d.

Variante	Fórmula	Notas
Raw	count	Sesgo a documentos largos
Booleana	1 si count > 0	Pierde intensidad
Log	$1 + \log_{10}(\text{count})$	Atenúa palabras hiperfrecuentes
Normalizada	$\text{count} / \max_t(\text{count})$	Compara documentos de distinta longitud

Inverse Document Frequency (IDF) — Spärck Jones (1972). Castiga términos que aparecen en muchos documentos (porque no sirven para distinguir):

$$\text{idf}(t, D) = \log \frac{N}{df(t)}$$

EN PALABRAS

"¿Qué tan raro y por lo tanto qué tan informativo es este término?"

- N = número total de documentos (p. ej. 4 obras).
- $df(t)$ = en cuántos documentos aparece el término t.
- Divides total / en-cuántos-aparece, y le aplicas logaritmo para suavizar.

EJEMPLO

Con $N = 4$:

- `good` aparece en las 4 obras → $\text{idf} = \log(4/4) = \log 1 = \mathbf{0}$ → "está en todas, no distingue nada".
- `battle` aparece solo en 3 → $\text{idf} = \log(4/3) = 0.12$ → algo de señal.
- Una palabra en 1 sola obra → $\log(4/1) = 0.60$ → señal fuerte, es una "firma" de ese documento.

*La variante **suavizada** (la que usa sklearn) es $\ln \frac{1+N}{1+df} + 1$: hace lo mismo pero suma 1 arriba y abajo para no dividir por cero, y suma 1 al final para que ningún término valga 0.*

TF-IDF = multiplicar cuántas veces aparece (TF, peso local) por qué tan raro es (IDF, peso global):

$$w_{t,d} = \mathrm{tf}(t,d) \cdot \mathrm{idf}(t,D)$$

> 💡 **IDEA CLAVE**
> Una palabra pesa mucho en un documento solo si **aparece bastante AQUÍ** *y* **es rara en el resto**. Si aparece en todos lados (idf≈0), su peso se aplasta aunque sea muy frecuente.

2.4 Normalización L2

Antes de comparar, ponemos todos los vectores del mismo “tamaño” (largo 1) para que un documento largo no gane solo por ser largo:

$$\hat{\mathbf{d}} = \frac{\mathbf{d}}{\|\mathbf{d}\|_2}, \qquad \|\mathbf{d}\|_2 = \sqrt{\textstyle\sum_i w_i^2}$$

> 🟦 **EN PALABRAS**
> $\|\mathbf{d}\|_2$ es el **largo** del vector (teorema de Pitágoras en muchas dimensiones: raíz de la suma de los cuadrados). Dividir el vector entre su largo lo deja de **largo 1** apuntando en la misma dirección.

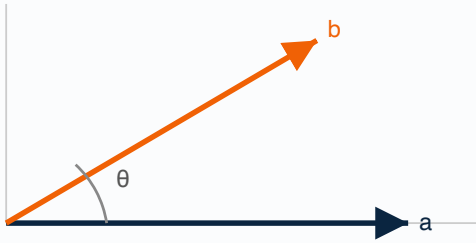
✏️ **EJEMPLO**

El vector $(3, 4)$ tiene largo $\sqrt{3^2 + 4^2} = \sqrt{25} = 5$. Normalizado: $(3/5,\ 4/5) = (0.6,\ 0.8)$, que ya mide 1. Tras L2 todos los documentos viven en una esfera de radio 1.

2.5 Similitud coseno

Mide el **ángulo** entre dos vectores: si apuntan al mismo lado, son parecidos.

$$\cos(\mathbf{a}, \mathbf{b}) = \frac{\mathbf{a} \cdot \mathbf{b}}{\|\mathbf{a}\|\,\|\mathbf{b}\|}$$



El coseno mide el ángulo θ: ángulo chico → vectores parecidos (cos→1); ángulo recto → sin relación (cos=0).

EN PALABRAS

Arriba va el **producto punto** $\mathbf{a} \cdot \mathbf{b} = \sum_i a_i b_i$ (multiplicas componente a componente y sumas); abajo, el producto de los **largos**. El resultado es el coseno del ángulo, entre -1 y 1.

EJEMPLO

$\mathbf{a} = (1, 0)$, $\mathbf{b} = (1, 1)$. Producto punto $= 1 \cdot 1 + 0 \cdot 1 = 1$. Largos: $\|a\| = 1$, $\|b\| = \sqrt{2} = 1.41$. Entonces $\cos = 1/(1 \cdot 1.41) = 0.71 \rightarrow$ ángulo de 45°, bastante parecidos.

$\cos\theta$	Interpretación
1	Misma dirección → idénticos en tema
0	En ángulo recto → nada que ver
-1	Opuestos (raro con TF-IDF, pesos ≥ 0)

Truco: *si ya normalizaste con L2 (largo 1), los de abajo valen 1 y el coseno es* ***solo el producto punto*** $\sum_i \hat{a}_i \hat{b}_i$*. Por eso L2 + coseno van juntos.*

¿Por qué coseno y no distancia normal (euclidiana)? La euclidiana mide cuán *lejos* están los puntos, y se deja engañar por el tamaño del documento (un libro vs. un párrafo). El coseno mira solo **hacia dónde apuntan** (el tema), no cuán largos son.

2.6 El "problema de la palabra dominante" (caso Shakespeare)

Cuando un término común (p. ej. `good`) tiene frecuencia mucho mayor que el resto, **domina** la norma L2 y arrastra el coseno hacia 1 para cualquier par de documentos:

> *Con frecuencias crudas,* $\cos(\text{Henry V}, \text{Julius Caesar}) = 0.999$ *— no por* `battle` *(temáticamente relevante) sino por el peso bruto de* `good`*.*

TF-IDF mitiga el efecto: multiplica `good` por una IDF baja y eleva el peso relativo de términos discriminativos como `battle`.

2.7 Reducción de dimensionalidad: PCA y t-SNE

PCA (Pearson 1901): base ortogonal que maximiza la **varianza explicada**.

1. Centrar $\tilde{X} = X - \mu$.
2. Covarianza $C = \tilde{X}^\top \tilde{X}/(n-1)$.
3. Diagonalizar $C = U \Lambda U^\top$.
4. Proyectar $Y = \tilde{X} \cdot U[:, :k]$ (para visualizar $k = 2$).

t-SNE (van der Maaten & Hinton 2008): preserva **vecindades locales** con un embedding probabilístico no lineal. Útil con muchos puntos; sin sentido con muy pocos (la perplejidad mínima supera la cardinalidad).

Aspecto	PCA	t-SNE
Tipo	Lineal	No lineal
Preserva	Estructura global (varianza)	Vecindades locales
Velocidad	Rápido	Lento
Reproducible	Sí (determinista)	No (estocástico, depende de `seed`)
Parámetro clave	`n_components`	`perplexity` (5–50)
Cuándo	Pocos puntos, ver ejes	Muchos puntos, ver clusters

2.7b Métricas de similitud / distancia

Métrica	Fórmula	¿Sensible a magnitud?	Uso típico
Coseno	$\frac{a \cdot b}{\|a\|\|b\|}$	No (solo dirección)	Texto / IR / embeddings
Producto punto	$a \cdot b$	Sí	Embeddings ya normalizados
Euclidiana (L2)	$\sqrt{\sum(a_i - b_i)^2}$	Sí	Clustering geométrico
Manhattan (L1)	$\sum\|a_i - b_i\|$	Sí	Robusta a outliers
Jaccard	$\frac{\|A \cap B\|}{\|A \cup B\|}$	— (conjuntos)	Bag-of-words binario

2.8 Ejemplos en Python: TF-IDF, coseno y PCA

(a) TF-IDF con sklearn (lo más usado):

```
from sklearn.feature_extraction.text import
        TfidfVectorizer
from sklearn.metrics.pairwise import
        cosine_similarity

docs = ["el gato come pescado", "el perro come
        carne", "el gato y el perro juegan"]
vec = TfidfVectorizer()
X = vec.fit_transform(docs)             #
        matriz dispersa (3 x |V|), ya L2-
        normalizada
print(vec.get_feature_names_out())
print(X.toarray().round(2))
print(cosine_similarity(X[0], X[1]))    #
        similitud doc0 vs doc1
```

(b) TF-IDF “a mano” con numpy (para entender la fórmula):

```
import numpy as np
# matriz término-documento (conteos):
        filas=términos, cols=docs
tf = np.array([[1,0,7,13],[114,80,62,89],
        [36,58,1,4],[20,15,2,3]], float)
N = tf.shape[1]
df = (tf > 0).sum(axis=1)
        # en cuántos docs aparece
idf = np.log((1+N)/(1+df)) + 1
        # idf suavizada (sklearn)
tfidf = tf * idf[:, None]
        # TF * IDF
tfidf /= np.linalg.norm(tfidf, axis=0)
        # normalización L2 por documento
print("idf:", idf.round(3))
```

(c) Similitud coseno + visualización PCA 2D:

```
from sklearn.decomposition import PCA
import matplotlib.pyplot as plt

cos = tfidf.T @ tfidf                    # tras
        L2, producto punto = coseno
P = PCA(n_components=2).fit_transform(tfidf.T)
plt.scatter(P[:,0], P[:,1])
for i, name in
        enumerate(["AYLI","TN","JC","HV"]):
    plt.annotate(name, P[i])
plt.title("Documentos en espacio TF-IDF (PCA
        2D)"); plt.show()
```

2.9 Mini-quiz — Semántica clásica

1. ¿Por qué se prefiere el coseno sobre la distancia euclidiana en IR?
2. ¿Qué le pasa a la IDF de un término que aparece en todos los documentos?
3. ¿Qué corrige TF-IDF respecto a TF cruda?
4. ¿Cuándo usar PCA y cuándo t-SNE?
5. Tras L2-norm, ¿qué relación hay entre producto punto y coseno?

PARTE III — Embeddings densos estáticos

3.1 Regresión logística en PLN (el clasificador base)

Clasificador discriminativo de referencia. Dado un vector de características x (p. ej. el TF-IDF de un texto), decide la probabilidad de que sea de la clase 1 (p. ej. "positivo"):

$$P(y = 1 \mid x) = \sigma(w^\top x + b), \qquad \sigma(z) = \frac{1}{1 + e^{-z}}$$

EN PALABRAS

Por partes:

- $w^\top x + b = w_1 x_1 + w_2 x_2 + \cdots + b$ → una **suma ponderada**: cada característica x_i se multiplica por su peso w_i (qué tan importante es) y se suma un sesgo b. Da un número de $-\infty$ a $+\infty$.
- σ (**sigmoide**) aplasta ese número al rango $[0, 1]$ para leerlo como **probabilidad**: muy negativos → ≈0, muy positivos → ≈1, el 0 → 0.5.

> ✏️ **EJEMPLO**
>
> Si la suma ponderada da $z = 2$, entonces $\sigma(2) = 1/(1 + e^{-2}) = 0.88$ → 88% de probabilidad de clase 1.

Se entrena ajustando los pesos w para que se equivoque lo menos posible (minimizar la **entropía cruzada**, que penaliza predecir con confianza algo falso); el término $\lambda\|w\|^2$ evita que los pesos se disparen (regularización).

Multiclase = softmax: cuando hay K clases (no solo 2), $P(y = k \mid x) = \dfrac{e^{w_k^\top x}}{\sum_j e^{w_j^\top x}}$ convierte K puntajes en K probabilidades que suman 1 (el exponencial hace todo positivo; dividir entre la suma normaliza). Es la capa final de Word2Vec y de casi toda red neuronal clasificadora.

3.2 Word2Vec — el cambio de paradigma

Mikolov et al. (2013). Reemplaza la matriz dispersa de TF-IDF por **vectores densos** (~100–300 dim) aprendidos por una red shallow que predice contextos.

Dos arquitecturas:

Variante	Predice	Mejor para
CBOW	la palabra dado su contexto	corpora pequeños, palabras frecuentes
Skip-gram	el contexto dada la palabra	corpora grandes, palabras raras

Skip-gram recorre el texto y, por cada palabra w_t, intenta **adivinar las palabras vecinas** (las que están a distancia $\leq c$, la "ventana"):

$$J(\theta) = -\frac{1}{T}\sum_{t=1}^{T}\sum_{-c\leq j\leq c,\, j\neq 0} \log P(w_{t+j} \mid w_t)$$

EN PALABRAS

Para cada palabra del corpus ($\sum_t$) y cada vecina dentro de la ventana ($\sum_j$), maximiza la probabilidad de acertar esa vecina. Si dos palabras tienen vecinos parecidos, acaban con vectores parecidos.

EJEMPLO

En `el ___ ladra`, tanto `perro` como `cachorro` encajan → sus vectores quedan cerca.

Negative Sampling (el truco que lo hace viable): calcular la probabilidad exacta exigiría comparar contra **todo** el vocabulario (millones de palabras) en cada paso. En vez de eso:

- empuja el vector de la palabra real y su vecino real a **acercarse**, y
- toma solo k palabras al azar ("negativas", que NO son vecinas) y las empuja a **alejarse**.

$$\log \sigma({v'_{w_O}}^\top v_{w_I}) + \sum_{i=1}^{k} \mathbb{E}_{w_i \sim P_n(w)} \left[\log \sigma(-{v'_{w_i}}^\top v_{w_I}) \right]$$

El primer término sube la similitud del par real; el segundo baja la de los k falsos. Con $k = 5$–20, el costo pasa de "revisar millones" a "revisar 20" por paso.

Subsampling de palabras frecuentes: palabras como `el`, `de`, `que` aparecen tantísimo que aportan poco. Se **tiran al azar** con probabilidad $P_{drop}(w) = 1 - \sqrt{t/f(w)}$ ($f(w)$ = qué tan frecuente es, $t \approx 10^{-5}$): cuanto más frecuente, más se descarta. Es el equivalente de la IDF para embeddings.

Propiedad emergente — analogías por aritmética con vectores. Como las relaciones

quedan codificadas como **direcciones** en el espacio, se pueden sumar y restar:

> $rey - hombre + mujer \approx reina$ · $París - Francia + Italia \approx Roma$

(quitas el componente “hombre” y agregas “mujer”, y caes cerca de “reina”).

3.3 Word2Vec / FastText con Gensim (el del taller)

(a) Entrenar ambos modelos:

```
from gensim.models import Word2Vec, FastText

# sg=1 → Skip-gram ; sg=0 → CBOW
w2v = Word2Vec(sentences=corpus,
        vector_size=300, window=5,
               min_count=5, sg=1, negative=10,
        sample=1e-3,
               epochs=5, workers=4, seed=42)

ft  = FastText(sentences=corpus,
        vector_size=300, window=5,
               min_count=5, sg=1, epochs=5,
        workers=4, seed=42)

w2v.save("w2v.model")                  #
        persistir
# w2v = Word2Vec.load("w2v.model")     #
        recargar
```

(b) Consultas semánticas:

```
w2v.wv["ciudad"]
        # vector (300,)
w2v.wv.most_similar("ciudad", topn=10)
        # vecinos por coseno
w2v.wv.similarity("rey", "reina")
        # coseno entre 2 palabras
w2v.wv.doesnt_match(["lunes","martes","perro"]
        )   # → 'perro' (el intruso)
w2v.wv.most_similar(positive=["rey","mujer"],
                    negative=["hombre"])
        # analogía → 'reina'
```

(c) FastText resuelve OOV y morfología:

```
"caminaríamos" in w2v.wv          # False →
        Word2Vec NO lo tiene
ft.wv["caminaríamos"]             # FastText
        SÍ: lo arma con n-gramas
ft.wv.most_similar("caminaríamos", topn=5)   #
        vecinos: caminar, caminé…
```

(d) Vector de un documento (promedio de embeddings):

```
import numpy as np
def doc_vector(tokens, wv):
    vs = [wv[t] for t in tokens if t in wv]
    return np.mean(vs, axis=0) if vs else
        np.zeros(wv.vector_size)

doc_vector(["la","ciudad","crece"], w2v.wv)
        # vector (300,) del documento
```

(e) Cargar embeddings preentrenados (sin entrenar):

```python
import gensim.downloader as api
glove = api.load("glove-wiki-gigaword-100")
        # 128 MB
glove.most_similar("king", topn=3)
```

3.4 GloVe — Global Vectors

Pennington, Socher & Manning (2014). Combina conteo global (LSA) con predicción local (Word2Vec). Aprende vectores tal que su **producto punto** se parezca a cuántas veces dos palabras **co-ocurren** (aparecen juntas) en todo el corpus:

$$w_i^\top \tilde{w}_j + b_i + \tilde{b}_j = \log X_{ij}$$

EN PALABRAS

X_{ij} = número de veces que las palabras i y j aparecen cerca en el corpus entero. GloVe ajusta los vectores $w_i, \tilde{w}_j$ (y unos sesgos b) para que su producto punto reproduzca el $\log$ de ese conteo. Si dos palabras co-ocurren mucho → vectores alineados. Word2Vec mira ventana por ventana; GloVe mira la **tabla global** de co-ocurrencias de una vez.

Aspecto	Word2Vec	GloVe
Señal	Pares locales (ventana)	Estadística global (matriz co-oc)
Empíricamente	Mejor en analogías sintácticas	Mejor en similitudes semánticas

3.5 FastText — subpalabras al rescate

Bojanowski et al. (2017). Representa cada palabra como **suma de n-gramas de caracteres** ($n \in [3, 6]$), rodeada por `<` y `>`. Ejemplo con `gato` (n-gramas de longitud 3):

> `gato` → `<ga`, `gat`, `ato`, `to>` + *la palabra entera* `<gato>`

El vector de la palabra es la **suma** de los vectores de sus pedacitos (n-gramas):

$$\mathbf{v}_w = \sum_{g \in \mathcal{G}_w} \mathbf{z}_g$$

EN PALABRAS

$\mathcal{G}_w$ = el conjunto de n-gramas de la palabra w; $\mathbf{z}_g$ = el vector de cada pedacito g. Sumas todos esos vectores y obtienes el vector de la palabra.

IDEA CLAVE

Por eso maneja palabras nuevas (OOV): aunque nunca haya visto `gatito`, conoce `gat`, `ato`, etc., y arma su vector sumando esos pedacitos.

Ventajas decisivas para el español:

- **OOV:** una palabra nueva se compone de sus n-gramas → siempre hay vector.
- **Morfología:** `caminar`, `caminé`, `caminaríamos` comparten n-gramas → vectores cercanos sin haberlos visto por separado. Por eso **FastText es el mejor de los dos** para español.

3.6 Tablas comparativas de embeddings densos

CBOW vs Skip-gram (las dos arquitecturas de Word2Vec):

Aspecto	CBOW	Skip-gram
Predice	La palabra ← su contexto	El contexto ← la palabra
Velocidad	Más rápido	Más lento
Palabras raras	Peor	**Mejor**
Corpus	Pequeño/mediano	Grande
Flag en Gensim	`sg=0`	`sg=1`

Word2Vec vs GloVe vs FastText:

Criterio	Word2Vec	GloVe	FastText
Año / autor	2013 Google	2014 Stanford	2017 Facebook
Señal de aprendizaje	Ventana local	Co-ocurrencia global	Ventana local + subpalabras
Unidad	Palabra	Palabra	Palabra + n-gramas
Maneja OOV	❌ No	❌ No	✅ Sí
Morfología	Débil	Débil	✅ Fuerte
Memoria entrenamiento	Baja	Alta (matriz co-oc)	Media
Fuerte en	Analogías sintácticas	Similitud semántica	Idiomas morfológicos (español)

> 💡 **IDEA CLAVE**
>
> Para **español**, FastText suele ganar: los sufijos verbales y de género/número comparten n-gramas, y nunca falla con palabras nuevas.

3.7 Mini-quiz — Embeddings densos

1. ¿Qué predice CBOW y qué predice Skip-gram?
2. ¿Qué problema resuelve el *negative sampling*?
3. ¿Por qué FastText maneja OOV y Word2Vec no?
4. ¿Qué diferencia la señal de GloVe de la de Word2Vec?
5. Explica `rey − hombre + mujer ≈ reina`.

PARTE IV — Embeddings contextuales y de oraciones

4.1 Limitación de los embeddings estáticos

Word2Vec/GloVe/FastText asignan **un único vector por palabra** independiente del contexto. Pero `banco` (institución) $\neq$ `banco` (asiento): hace falta **polisemia** → embeddings **contextuales**.

Aspecto	Estático (Word2Vec/GloVe/FastText)	Contextual (BERT/BETO)
Vectores por palabra	**Uno fijo**	**Uno por contexto**
Polisemia (`banco`)	❌ mismo vector	✅ vector distinto según frase
Costo	Bajo (lookup en tabla)	Alto (pasar por la red)
Entrenamiento	Ventana local	Transformer + atención
Ejemplo de uso	Similitud de palabras	Clasificación, NER, QA

4.2 BERT (y BETO)

Devlin et al. (2018). Marcó la transición a la era Transformer.

Componente	Detalle
Arquitectura	Stack de **encoders** Transformer (12 o 24 capas)
Preentrenamiento	**MLM** (predecir tokens enmascarados) + **NSP** (¿frase siguiente?)
Tokens especiales	`[CLS]` , `[SEP]`
Tokenización	WordPiece
Salida	Embedding **contextual** por token (768 o 1024 dim)

El embedding de `[CLS]` representa la oración completa para clasificación. **BETO** es la versión española (`dccuchile/bert-base-spanish-wwm-cased`). Discriminativo (clasifica/predice), no generativo.

Familia BERT (variantes que conviene conocer):

Modelo	Aporte clave
BERT (2018)	Encoder bidireccional, MLM + NSP
RoBERTa (2019)	BERT mejor entrenado, sin NSP
DistilBERT (2019)	Destilado: 40% más chico, 60% más rápido
ALBERT (2019)	Parámetros compartidos, menor footprint
BETO	BERT entrenado en español (WordPiece, cased)

Embedding contextual con BETO en Python:

```
import torch
from transformers import AutoTokenizer,
        AutoModel
tok =
        AutoTokenizer.from_pretrained("dccuchi
        le/bert-base-spanish-wwm-cased")
model =
        AutoModel.from_pretrained("dccuchile/b
        ert-base-spanish-wwm-cased")

enc = tok("El banco del río",
        return_tensors="pt")
with torch.no_grad():
    out = model(**enc)
cls_vec = out.last_hidden_state[:, 0]      #
        embedding del token [CLS] (768,)
print(cls_vec.shape)                        #
        torch.Size([1, 768])
```

4.3 Sentence-BERT (SBERT) y sentence-transformers

Reimers & Gurevych (2019). Red siamesa sobre BERT entrenada con *contrastive/triplet loss* sobre pares de oraciones. Produce embeddings de **oración** comparables por coseno **sin** pasar pares por el modelo (≈104× más rápido que BERT crudo en STS, *Semantic Textual Similarity*: medir cuán parecidas son dos frases).

```
from sentence_transformers import
        SentenceTransformer
from sklearn.metrics.pairwise import
        cosine_similarity
m = SentenceTransformer("all-MiniLM-L6-v2")
e = m.encode(["el gato duerme", "el felino
        descansa"], normalize_embeddings=True)
cosine_similarity([e[0]], [e[1]])   # alta
        similitud aunque no compartan palabras
```

Los cuatro modelos del taller:

Modelo	Dim	Idioma	Uso
`all-mpnet-base-v2`	768	EN (monolingüe)	Alta precisión, más lento
`all-MiniLM-L6-v2`	384	EN (monolingüe)	Rápido y eficiente
`paraphrase-multilingual-MiniLM-L12-v2`	384	multilingüe	Búsqueda/similitud multilingüe
`BAAI/bge-m3`	1024	multilingüe	Recuperación, **excelente en español**

Conclusión del taller: los multilingües (`paraphrase-multilingual`, `bge-m3`) recuperan fragmentos en español mejor que los monolingües entrenados en inglés. `bge-m3` da el mejor balance.

4.4 Búsqueda semántica y RAG (chunking + coseno)

Pipeline de recuperación semántica sobre documentos (el punto 3 del taller), en dos fases:

Indexación (una vez): PDF → texto (pymupdf4llm) → *chunking* → embeddings → índice de vectores.

Consulta (cada pregunta): texto de la consulta → embedding → similitud coseno contra el índice → chunk más similar.

Chunking con `RecursiveCharacterTextSplitter` (`chunk_size=1000`, `chunk_overlap=200`): parte el texto respetando límites naturales (`\n\n`, `\n`, `.`, ` `). El *overlap* evita cortar ideas a la mitad. Buen chunking = mejor recuperación.

Pipeline completo de recuperación semántica en Python:

```
import glob, numpy as np, pymupdf4llm
from langchain_text_splitters import
        RecursiveCharacterTextSplitter
from sentence_transformers import
        SentenceTransformer
from sklearn.metrics.pairwise import
        cosine_similarity

# 1) Cargar PDFs y trocear
splitter = RecursiveCharacterTextSplitter(
    chunk_size=1000, chunk_overlap=200,
    separators=["\n\n", "\n", ". ", " ", ""])
chunks = []
for pdf in glob.glob("pdf-coding/*.pdf"):
    texto = pymupdf4llm.to_markdown(pdf)
    chunks += [c for c in
        splitter.split_text(texto) if len(c) >
        30]

# 2) Indexar: un embedding por chunk
        (normalizado para usar coseno)
modelo = SentenceTransformer("BAAI/bge-m3")
emb = modelo.encode(chunks,
        normalize_embeddings=True)

# 3) Consultar: vectorizar la pregunta y
        buscar el chunk más similar
q = modelo.encode(["¿objetivos del programa de
        maestría?"],
                  normalize_embeddings=True)
sims = cosine_similarity(q, emb)[0]
mejor = int(np.argmax(sims))
print(f"cos={sims[mejor]:.3f}\n{chunks[mejor]
        [:300]}")
```

Esto es la base de **RAG** (Retrieval-Augmented Generation): recuperar los chunks más relevantes por coseno y dárselos a un LLM como contexto.

> 💡 **IDEA CLAVE**
>
> Normaliza los embeddings (`normalize_embeddings=True`) y el coseno se reduce a un producto punto → puedes indexar con FAISS para buscar en millones de chunks en milisegundos.

4.5 NER con BioBERT (el dataset biomédico del taller)

NER (Named Entity Recognition): etiquetar cada token con su tipo de entidad. El dataset BioBERT tiene 29 etiquetas biomédicas en esquema **IOB**:

- `B_XXX` = **B**eginning (inicio de entidad tipo XXX).
- `I_XXX` = **I**nside (continuación de la entidad).
- `O` = fuera de toda entidad.

Ejemplo real (`train[0]`) — cada palabra con su etiqueta:

Palabra	Etiqueta	Significado
Abuela	`B_FAMILY`	inicio de entidad “familiar”
materna	`I_FAMILY`	continúa la entidad familiar
con	`O`	fuera de entidad
cancer	`B_CANCER_CONCEPT`	inicio de concepto de cáncer
de mama	`I_CANCER_CONCEPT`	continúa el concepto
70	`B_QUANTITY`	cantidad
años	`B_METRIC`	unidad/métrica

Cargar el dataset y mostrar la anotación (Python):

```
from datasets import load_dataset
ds = load_dataset("json", data_files={
    "train": "Biobert/data56/train.json",
    "validation": "Biobert/data56/valid.json",
    "test": "Biobert/data56/test.json"})

TAGS = ["B_CANCER_CONCEPT","B_CHEMOTHERAPY",
        ...,"O"]   # 29 etiquetas
ej = ds["train"][0]
for palabra, tid in zip(ej["sentencia"],
        ej["tag"]):
    print(f"{palabra:12s} {TAGS[tid]}")
```

Alinear etiquetas con subtokens de WordPiece (clave en NER real):

```
from transformers import AutoTokenizer
tok =
        AutoTokenizer.from_pretrained("dccuchi
        le/bert-base-spanish-wwm-cased")

enc = tok(ej["sentencia"],
        is_split_into_words=True,
        truncation=True)
labels = []
for wid in enc.word_ids():                  #
        mapea cada subtoken a su palabra
    if wid is None:         labels.append(-100)
        # [CLS]/[SEP] → se ignoran
    else:
        labels.append(ej["tag"][wid]) #
        subtoken hereda la etiqueta
print(list(zip(tok.convert_ids_to_tokens(enc["
        input_ids"]), labels)))
```

⚠ **OJO**

Como WordPiece parte `70` → `7 ##0`, hay **más subtokens que palabras**. Se usa `word_ids()` para que cada subtoken herede la etiqueta de su palabra, y los especiales `[CLS]` / `[SEP]` se marcan con `-100` (PyTorch los ignora en la pérdida).

4.6 Mini-quiz — Contextuales y oraciones

1. ¿Por qué BERT resuelve la polisemia y Word2Vec no?
2. ¿Qué tarea de preentrenamiento usa BERT además de NSP?
3. ¿Qué ventaja tiene SBERT sobre BERT crudo para comparar oraciones?
4. ¿Para qué sirve el `chunk_overlap` en el chunking?
5. En IOB, ¿qué significan `B_`, `I_` y `O`?

PARTE V — Tabla maestra de representaciones

Resumen de **todas** las formas de representar texto vistas en el curso, de la más simple a la más potente:

Representación	Tipo	Dim típica	OOV	Contexto	Captura	Costo
One-hot	Dispersa	$\|V\|$	❌	❌	Identidad de palabra	Mínimo
TF-IDF	Dispersa	$\|V\|$	❌	❌	Relevancia en documento	Bajo
Word2Vec	Densa estática	100–300	❌	❌	Semántica por co-ocurrencia	Medio
GloVe	Densa estática	100–300	❌	❌	Co-ocurrencia global	Medio

Representación	Tipo	Dim típica	OOV	Contexto	Captura	Costo
FastText	Densa estática	100–300	✅	❌	Semántica + morfología	Medio
BERT/BETO	Densa contextual	768–1024	✅ (WordPiece)	✅	Polisemia, sintaxis	Alto
SBERT/bge-m3	Densa de oración	384–1024	✅	✅	Significado de frase	Alto

IDEA CLAVE

Regla mental: **dispersa** (TF-IDF) = cuenta palabras; **densa estática** (Word2Vec/FastText) = una foto fija del significado; **contextual** (BERT) = el significado según la frase; **de oración** (SBERT) = comparar frases enteras por coseno.

PARTE VI — Formulario de

bolsillo (cheat sheet)

Concepto	Fórmula	Qué dice (en simple)
IDF (clásica)	$\log(N/df)$	Qué tan rara/informativa es una palabra
IDF (suavizada)	$\ln \frac{1+N}{1+df} + 1$	Igual, sin dividir por cero
TF-IDF	$\mathrm{tf} \cdot \mathrm{idf}$	Frecuente aquí **y** rara en el resto
Norma L2	$\sqrt{\sum w_i^2}$	El “largo” del vector
Coseno	$\frac{a \cdot b}{\|a\|\|b\|}$	El ángulo: ¿apuntan al mismo lado?
L2 ⇒ coseno	$\|a - b\|^2 = 2 - 2\cos$	Tras L2, distancia y coseno son lo mismo
Sigmoide	$\frac{1}{1+e^{-z}}$	Aplasta un número a probabilidad $[0, 1]$
Softmax	$\frac{e^{z_k}}{\sum_j e^{z_j}}$	K puntajes → K probabilidades que suman 1
WordPiece	$\frac{P(AB)}{P(A)P(B)}$	¿Vale la pena pegar estas 2 piezas?
FastText	$v_w = \sum_g z_g$	Palabra = suma de sus pedacitos

Concepto	Fórmula	Qué dice (en simple)
Subsampling	$1 - \sqrt{t/f(w)}$	Tira palabras demasiado frecuentes

Decisiones rápidas (¿qué uso?):

Caso	Recomendación
Baseline, pocas oraciones, vocab fijo	TF-IDF + coseno
Similitud de palabras, idioma morfológico	FastText
Manejar OOV	FastText
Polisemia / contexto	BERT / BETO
Comparar oraciones, búsqueda semántica, RAG	SBERT / bge-m3 + coseno
Clasificación SOTA con recursos	Fine-tune BERT/RoBERTa

PARTE VII — Banco de preguntas con respuestas

Tokenización

1. **WordPiece vs BPE:** BPE fusiona el par *más frecuente*; WordPiece el que *maximiza la verosimilitud* $P(AB)/(P(A)P(B))$.
2. `##` **y** `_`**:** `##` marca subpalabra interna en WordPiece (`##ization`); `_` marca inicio de palabra (espacio) en SentencePiece.
3. **Subpalabra > palabra:** evita vocabularios enormes y el problema OOV manteniendo secuencias cortas.
4. `[CLS]`**:** token inicial cuyo embedding agregado se usa como representación de la secuencia para clasificación.
5. **Unigram:** puede muestrear segmentaciones alternativas → regularización / *data augmentation*.

Semántica clásica

6. **Coseno vs euclidiana:** el coseno mide dirección (robusto a la longitud del documento); la euclidiana se sesga con la magnitud.

7. **IDF de término ubicuo:** ≈ 0 (clásica) o 1 (suavizada) $\rightarrow$ poca señal discriminativa.
8. **TF-IDF corrige:** el sesgo de palabras frecuentes poco informativas (problema de la palabra dominante).
9. **PCA vs t-SNE:** PCA = lineal/global/rápido/reproducible; t-SNE = no lineal/local/lento/estocástico, para muchos puntos.
10. **L2** $\Rightarrow$ producto punto = coseno (todos los vectores en la esfera unitaria).

Embeddings densos

11. **CBOW/Skip-gram:** CBOW predice la palabra desde el contexto; Skip-gram predice el contexto desde la palabra.
12. **Negative sampling:** evita el softmax sobre todo el vocabulario muestreando k negativos $\rightarrow O(|V|) \rightarrow O(k)$.
13. **FastText OOV:** compone la palabra desde n-gramas de caracteres, así que siempre produce vector; Word2Vec asigna un vector único por palabra completa.
14. **GloVe vs W2V:** GloVe usa estadística global de co-ocurrencia; Word2Vec usa pares locales de ventana.

15. **Analogía:** las relaciones semánticas son **direcciones lineales** en el espacio, así `rey−hombre+mujer` cae cerca de `reina`.

Contextuales y oraciones

16. **BERT y polisemia:** genera un vector **por contexto**, así `banco` tiene representación distinta según la oración.
17. **Preentrenamiento BERT:** MLM (predecir tokens enmascarados) + NSP (frase siguiente).
18. **SBERT:** produce embeddings de oración comparables por coseno sin reprocesar pares → órdenes de magnitud más rápido para búsqueda/STS.
19. `chunk_overlap`**:** solapa fragmentos para no cortar una idea a la mitad y preservar contexto en la frontera.
20. **IOB:** `B_` inicio de entidad, `I_` continuación, `O` fuera de entidad.

PARTE VIII — Ejemplos resueltos (material de quiz)

VII.A Caso Shakespeare: TF-IDF + coseno paso a paso

Corpus clásico de Jurafsky & Martin (Fig. 6.2). Obras: **AYLI** = *As You Like It*, **TN** = *Twelfth Night*, **JC** = *Julius Caesar*, **HV** = *Henry V*. Matriz término–documento (conteos):

término	AYLI	TN	JC	HV
battle	1	0	7	13
good	114	80	62	89
fool	36	58	1	4
wit	20	15	2	3

Paso 1 — df e idf (4 documentos, $N = 4$):

Término	df	idf clásica $\log_{10}(N/df)$	idf suavizada $\ln\frac{1+N}{1+df}+1$
battle	3	0.1249	**1.2231**
good	4	0	1.0000
fool	4	0	1.0000
wit	4	0	1.0000

Solo `battle` (df=3 < N) tiene idf > 1; los términos ubicuos quedan en idf=1. **Por eso `battle` es el único cuyo peso cambia entre raw y TF-IDF.**

Paso 2 — coseno con TF-IDF (matriz de similitud entre obras):

	AYLI	TN	JC	HV
AYLI	1.000	0.950	0.942	0.945
TN	0.950	1.000	0.807	0.817
JC	0.942	0.807	1.000	**0.9988**
HV	0.945	0.817	**0.9988**	1.000

Paso 3 — coseno con frecuencias crudas: $\cos(\mathrm{HV}, \mathrm{JC}) = \mathbf{0.9991}$ (el famoso ≈0.999 del enunciado).

Respuestas del quiz:

- **Obra más similar a `Henry V`:** `Julius Caesar` (0.9988 con TF-IDF; 0.9991 raw). El orden `JC > AYLI > TN` **no cambia** entre raw y TF-IDF.
- **¿Por qué HV ≈ JC = 0.999 en crudo?** Porque `good` (89 y 62) **domina la norma L2** de ambos vectores: la proporción $good/\|d\| \approx 0.99$, así que los vectores quedan casi paralelos al eje `good` y el coseno colapsa: $\cos(a,b) \approx \frac{a_2 b_2}{|a_2||b_2|} = 1$, insensible al resto.
- **¿Qué corrige TF-IDF?** Penaliza términos de alto df (ubicuos, poca señal) y premia los de bajo df (firmas del documento). Aquí casi no cambia el ranking porque 3 de 4 términos están en *todos* los documentos (idf=1); con un corpus más diverso sí reordenaría. **TF-IDF re-pesa, no es bala de plata.**
- **PCA 2D:** sobre las columnas (documentos), varianza explicada **PC1 ≈ 96.4 %**, PC2 ≈ 3.4 % (acumulado ≈ 99.8 %).

VII.B Comparación de tokenizadores (13 modelos sobre Spanish Billion Words)

Tokens producidos por 5 sentencias de prueba, por algoritmo × tamaño de vocabulario:

Modelo	TOTAL tokens (5 sent.)
bpe-10000 / wp-10000	66
bpe-20000 / wp-20000	55
bpe-30000 / wp-30000	**50**
word-10000	55
word-20000 / 30000	54
char-10000/20000/30000	**409**

Lecturas clave (típicas de quiz):

- `char` **produce ~7× más tokens** que `bpe–30000` → costo de atención $O(n^2)$ inviable para inputs largos.
- **Subir vocab 10k→30k** reduce ~25 % los tokens en BPE/WordPiece; en `word` la ganancia se aplana (la cola frecuente ya está cubierta); en `char` **no aporta nada** (alfabeto

cubierto con <200 entradas → conteo de tokens no varía con vocab_size).

- **BPE y WordPiece con el mismo vocab dan conteos casi idénticos** (comparten algoritmo base); la diferencia es la presencia de `<sep>` / `<cls>` en el inventario.
- `word` empata a BPE en compresión sobre dominio cubierto, pero degrada con OOV (→ `<unk>`).

Símbolos especiales en SentencePiece (`user_defined_symbols`): `<cls>` y `<sep>` reciben un ID dedicado y se preservan como unidad al tokenizar (`sp.piece_to_id('<cls>')` → ID fijo).

Trade-off práctico: LLM multilingüe grande → BPE/Unigram vocab 32k–256k · BERT mono-idioma → WordPiece 30k · transcripción/fonética → char · baseline con vocab cerrado → word 20k.

Referencias

- Jurafsky & Martin (2024). *Speech and Language Processing* (3.ª ed., draft), caps. 5, 6, 11.
- Sennrich et al. (2016) BPE · Schuster & Nakajima (2012) WordPiece · Kudo (2018) Unigram/SentencePiece.
- Salton et al. (1975) VSM · Spärck Jones (1972) IDF · Salton & Buckley (1988) term weighting.
- Mikolov et al. (2013) Word2Vec · Pennington et al. (2014) GloVe · Bojanowski et al. (2017) FastText.
- Devlin et al. (2018) BERT · Reimers & Gurevych (2019) Sentence-BERT.